

Quine

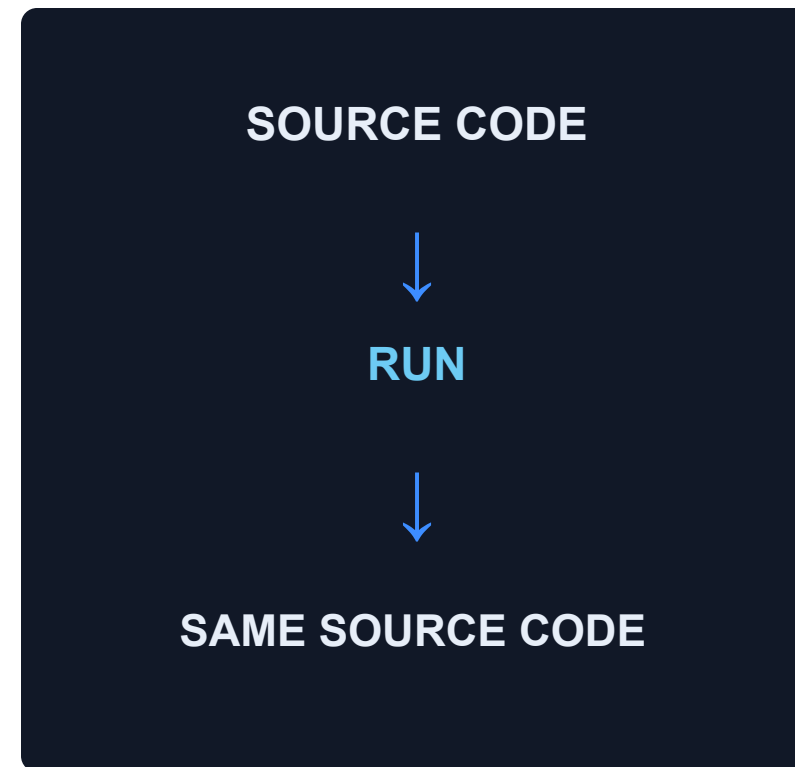
自分自身を出力する
プログラムは、どう作る？

自己参照を前提にせず、
「引用」と「テンプレート」で組み立てる。

Quineは、読み込まずに自分自身を出力する

Quineとは、入力やソースファイルの読み取りをせず、
自分自身のソースコードを出力するプログラムです。

ポイントは、外部から自分の内容を取得しないこと。
プログラムの内部だけで、同じソースを組み立てます。



INPUT = 0

SOURCE READ = 0

OUTPUT = SOURCE

出力はソースと1文字も変わらない

```
terminal

$ cc quine.c -o quine

$ ./quine > output.c

$ diff -u quine.c output.c

  差分出力なし

$ echo $?

0
```

実行後

quine.c == output.c

0

diff の終了コード

差分なし
= ソースと完全一致

自分のファイルを読むだけでは Quineではない

C / EXCERPT

```
1  int c;  
2  FILE *fp = fopen(__FILE__, "r");  
3  while ((c = fgetc(fp)) != EOF) putchar(c);
```

[SOURCE FILE](#) → [COPY](#)

これは何をしている？

自分のソースを
外部ファイルから読み取り、
そのままコピーしている

Quine 内部から組み立てる

File copy 外部から読み取る

同じ複雑さのものを、どう複製する？

NAIVE QUESTION

作られる工場そのもの

+ 複製するための仕組み

= 作る側はもっと複雑？

素朴な疑問

作る側の方が
複雑になる？

では、同じ複雑さの工場は
作れないのだろうか？

ソースを直接書き込むと、無限に膨らむ

```
printf("ソースコード");
```



出力には、その printf 自身も必要



さらに、その printf を出力する printf も必要 → ...

プログラム全体を文字列へ入れるだけでは、終わらない。

“Write this sentence.” は、自己参照を前提にする

INSTRUCTION

Write this sentence.

EXECUTE
↓
SAME
SENTENCE

01 一見、自己複製できる

Write this sentence.

02 でも “this sentence” とは？

文全体への参照

「自分のコード」を指す機能を
最初から使っている

まず、Hello Worldで引用を試す

ACTION

Write the following twice,
the second time in quotes.

TEMPLATE

"Hello World"

OUTPUT 1

Hello World

OUTPUT 2

"Hello World"

TEMPLATEを、ACTIONそのものに置き換える

ACTION

Write the following twice,
the second time in quotes.

TEMPLATE

"Write the following twice,
the second time in quotes."

OUTPUT 1

Write the following twice,
the second time in quotes.

OUTPUT 2

"Write the following twice,
the second time in quotes."

8枚目と同じ形式のまま、TEMPLATEだけをACTIONへ置き換える

show は引用符とエスケープを同時に作る

HASKELL

```
1  main = putStrLn (s ++ show s)

2  where s = "main = putStrLn (s ++ show s)\n  where s = "
```

s ++ show s = 元のソースコード全体

01 ACTION

s をそのまま出し、
show s を続ける。

02 TEMPLATE

s は、プログラム前半を
保持する文字列データ。

```
s = "print\n"
show s = "\"print\\n\""
```

まず、CのQuine全体を眺める

quine.c

表示上のみ折り返し

```
1  #include <stdio.h>

2  int main(void){char*s="#include <stdio.h>%cint main(void){char*s=
    ↳ %c%s%c;printf(s,10,34,s,34,10);}%c";
    ↳ printf(s,10,34,s,34,10);}
```

雛形

文字列 s がソースの形を持つ

埋め込み

printf が s 自身と記号を差し込む

ここでは全体像だけ確認する。次は、読みやすい別実装で s の中身を追う。

読みやすい別実装では、s に雛形全文を入れる

quine.c

表示上のみ折り返し

```
1  #include <stdio.h>

2  int main(void)

3  {

4      char *s = "#include <stdio.h>%cint main(void)%c{%c

↳      char *s = %c%s%c;%c    printf(s, 10, 10, 10,

↳      34, s, 34, 10, 10, 10);%c}%c";

5      printf(s, 10, 10, 10, 34, s, 34, 10, 10, 10);

6  }
```

01

TEMPLATE

s に、ソース全体の雛形を
書式文字列として保持

02

ACTION

printf が改行・引用符・
s 自身を空欄へ挿入

compile → run → diff = 0

Cでは、引用符と改行を自分で差し込む

```
printf(s, 10, 10, 10, 34, s, 34, 10, 10, 10);
```

雛形の空欄

渡す値

できるもの

%s

s

雛形自身の文字列

%c

10

改行

%c

34

引用符 "

Cでは 34 と 10 で引用符と改行を差し込み、文字列中の \ を避けている。

同じ `s` を2つの形で出力する

HASKELL

`show` が引用符とエスケープを担当

```
main = putStrLn (s ++ show s)
  where s = "main = putStrLn (s ++ show s)\n  where s = "
```

C

`printf` の構造を省略表示

```
char *s = "...%c...%c%s%c...%c";
printf(s, 10, 34, s, 34, 10);
```

`show s`

quote + escape を自動生成

`%s / %c`

`s`・引用符・改行を手動で合成

Quineの核心は、同じ `s` を「雛形」と「データ」にすること

01 同じテンプレート `s` を使う

`s` を、出力の雛形と埋め込むデータの二役に使う

02 `TEMPLATE:s` から出力を組み立てる

Haskellでは前半、Cでは書式文字列として使う

03 `QUOTED:s` のコード表現を埋め込む

`show` または引用符・改行で、`s` 自身をコード化する

TEMPLATE: `s`

+

QUOTED: `quote(s)`

= SOURCE

同じ `s` を「出力の雛形」と「埋め込むデータ」の二役に分けて組み合わせる。